

Scaling Up: Function Approximation in Reinforcement Learning

Generalization, semi-gradients, and the stability boundary

Learning objectives

By the end of this lesson, you should be able to:

- 01 — Quantify why tabular Q-learning fails in large or continuous spaces.
- 02 — Represent action values as a linear function of state-action features.
- 03 — Design normalized features that encode useful similarity.
- 04 — Derive the semi-gradient Q-learning weight update.
- 05 — Compute a complete numerical TD update by hand.
- 06 — Explain the deadly triad and identify stability evidence.

Approximation trades exact storage for generalization

A parameterized Q function shares statistical strength across state-action pairs but couples their errors.

Generalization

Updating one experience changes every prediction that uses overlapping features.

- Memory depends on parameter count, not the number of states.
- Features define which situations share value information.
- Approximation introduces bias and possible instability.

Linear action-value approximation

$$Q_{\hat{}}(s,a;w) = w^T f(s,a) = \sum_i w_i f_i(s,a)$$

- $f(s,a)$ is a d -dimensional feature vector.
- w contains d learned weights.
- One-hot features recover a tabular Q function.
- Dense shared features create generalization.

ENGINEERING IMPLICATION

Feature scale and semantics become part of the agent contract.

A table memorizes; a feature model shares

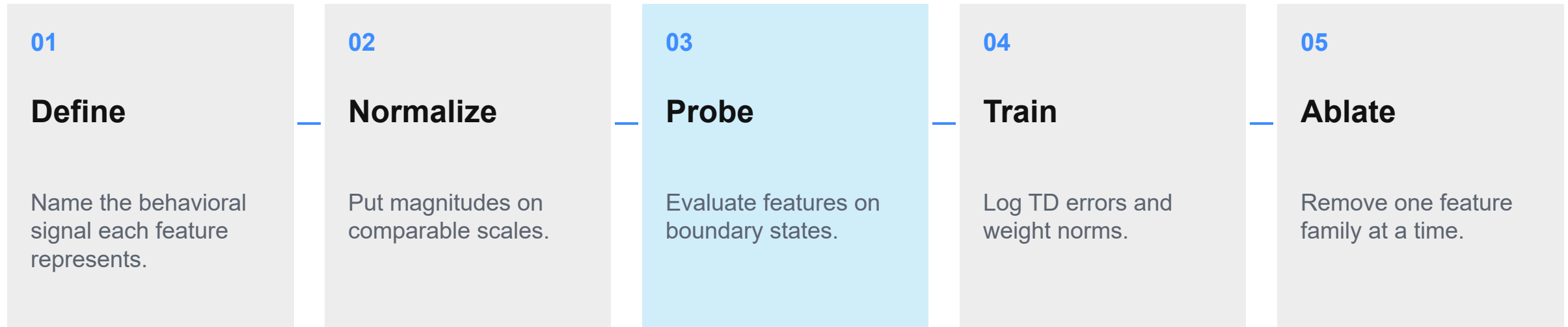
Tabular Q

- One value per state-action pair.
- No transfer to unseen pairs.
- Exact representation in finite tables.
- Memory grows as $|S||A|$.

Linear approximation

- One dot product per action.
- Transfer through shared features.
- Possible approximation bias.
- Memory grows with d .

Feature design is a modeling workflow



A feature is justified by predictive evidence, not by an intuitive name.

Semi-gradient Q-learning with linear features

```
initialize w = zeros(d)
for each episode:
    s = env.reset()
    while not done:
        a = epsilon_greedy(Q_hat(s, ., w))
        s_next, r, done = env.step(a)
        target = r if done else
            r + gamma * max_a Q_hat(s_next, a, w)
        delta = target - Q_hat(s, a, w)
        w = w + alpha * delta * features(s, a)
        s = s_next
```

Implementation contract

- Treat target as fixed.
- Mask terminal bootstrap.
- Cache the feature vector.
- Monitor finite weights.

Worked update: one transition changes three weights

SCENARIO

**$w=[0.4,-0.2,0.1]$, $f(s,a)=[1,0.5,2]$,
reward=1, gamma=0.9, max next
 $Q=1.5$, alpha=0.1.**

- Current Q is $0.4-0.1+0.2=0.5$.
- Target is $1+0.9(1.5)=2.35$.
- TD error is $2.35-0.5=1.85$.
- Add 0.185 times the feature vector to w .

The new weights are $[0.585,-0.1075,0.47]$, and every overlapping prediction changes.

Useful grid features encode distinct signals

Geometry

Where am I?

- Normalized row and column.
- Distance to goal.
- Distance to hazards.
- Wall adjacency indicators.

Action relation

What will this move do?

- Alignment with goal direction.
- Collision indicator.
- Progress delta.
- Action one-hot encoding.

Quality checks

Can it be trusted?

- Finite and bounded values.
- Stable scale across maps.
- Cheap computation.
- Ablation evidence.

Checkpoint: identify shared updates

CHECKPOINT

Two state-action pairs have feature vectors $[1,1,0]$ and $[1,0,1]$. Which prediction changes after updating the first pair?

01

Write the weight increment.

02

Take its dot product with the second vector.

03

State which shared coordinate causes transfer.

A semi-gradient freezes the bootstrap target

Prediction side

- Differentiate $Q_{\text{hat}}(s,a;w)$.
- Gradient is $f(s,a)$ for a linear model.
- Update parameters each step.
- Directly controls current prediction.

Target side

- Contains a next-state prediction.
- Treated as a constant for this step.
- Not differentiated through.
- Moves again after parameters change.

Deriving the weight update

$$L=0.5(y-Q_{\hat{}})^2; \text{delta}=y-Q_{\hat{}}; w \leftarrow w + \alpha \text{delta} \text{grad}_w Q_{\hat{}}$$

- $y=r+\gamma \max_a Q_{\hat{}}(s_{\text{next}},a)$ for nonterminal steps.
- The negative loss gradient is $\text{delta} \text{grad}_w Q_{\hat{}}$.
- For a linear model, $\text{grad}_w Q_{\hat{}}=f(s,a)$.
- Terminal y equals reward only.

ENGINEERING IMPLICATION

The semi-gradient update resembles supervised regression onto a moving TD target.

One-hot features recover the tabular rule

SCENARIO

A feature vector has one active coordinate for exactly one pair (s,a).

- The dot product selects one weight.
- Its gradient is one at that coordinate and zero elsewhere.
- The update changes only that selected weight.
- The result matches $Q(s,a) \leftarrow Q(s,a) + \alpha \delta$.

Tabular Q-learning is a linear approximator with a very large one-hot feature basis.

The deadly triad marks a risk boundary

Approximation

Shared parameters

- Couples predictions.
- Creates representation error.
- May amplify updates.
- Enables scale.

Bootstrapping

Learn from estimates

- Uses a moving target.
- Propagates value quickly.
- Can propagate error.
- Reduces full-return delay.

Off-policy

Learn a different policy

- Behavior and target differ.
- Supports exploration reuse.
- Changes sampling distribution.
- Completes the triad.

Checkpoint: diagnose divergence

CHECKPOINT

Training return rises while weight norm and absolute TD error explode. Is high return alone enough to accept the agent?

01

Name two stability metrics.

02

Check multiple seeds.

03

Separate evaluation from exploratory return.

Instrument approximation before tuning

A scalable agent needs observability for representation error and numerical stability.

- Log returns, TD-error quantiles, feature ranges, and weight norms.
- Test terminal masking and feature dimensions directly.
- Compare against a tabular oracle on a small discretized environment.

Evidence

Learning curves alone cannot distinguish lucky behavior from a stable value function.

Key takeaways

01 ————— **Linear Q functions generalize through shared features.**

02 ————— The semi-gradient update freezes the TD target.

03 ————— One-hot features reveal the table as a special case.

04 ————— The deadly triad motivates explicit stability monitoring.

Next: Deep Q-networks replace hand-designed linear features with learned representations.

Check your understanding

Q1

Compute a linear Q value from w and f .

Q2

Derive the semi-gradient sign.

Q3

Explain why feature normalization matters.

Q4

Show the one-hot tabular correspondence.

Q5

Design a divergence-detection test.